



Neumann János Egyetem
Műszaki és Informatikai
Kar

Cowrie SSH Honeyypot

Cowrie SSH Honeyypot

Tartalomjegyzék

BEVEZETÉS	3
1. ELMÉLETI HÁTTÉR	4
2. SHELL EMULÁCIÓ ÉS PROXY MÓD KÖZÖTTI KÜLÖNBSÉG	5
3. ANYAG ÉS MÓDSZER.....	6
3.1. ANYAG	6
3.2. MÓDSZER	6
3.2.1. Hálózati infrastruktúra	6
3.2.2. Cowrie SSH Honeypot telepítése és konfigurálása.....	7
3.2.3. Naplóadatok kinyerése és feldolgozása.....	10
4. HIBÁK ÉS TANULSÁGOK	15
5. TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK.....	17
6. ÖSSZEGZÉS.....	19
ÁBRAJEGYZÉK	20

Bevezetés

Projektfeladatként egy SSH honeypot felállítását választottam. Ennek fő oka, hogy a téma fontos részét képezi a kibervédelemnek, hisz a honeypot-ok segítségével tudok információkat gyűjteni a leggyakrabban használt támadási mintákról. Sokat segít így a különböző támadási típusok kielemezésében és az ellenük készített védelmek fejlesztésében is.

A projekt elkezdésekor szembesülnöm kellett azzal, hogy nem teljesen voltam eddigiekben tisztában ezzel a technológiával. Az elején egy Shell emulációt akartam alkalmazni, de később megtudtam, hogy egy proxy alapú honeypot sokkal jobban tükrözi a valóságban is használt és felállított honeypotokat, míg a Shell emuláció inkább egy otthoni kipróbálásra készült, hiszen sokkal könnyebben felállítható.

1. Elméleti háttér

Az SSH, vagyis Secure Shell egy olyan hálózati protokoll, amely távoli bejelentkezést és parancsfuttatást tesz lehetővé Linux rendszereken. Mivel az SSH gyakran adminisztrációs célra van használva, ezért kiemelt célpont lehet támadók számára. Ha egy támadó hozzáférést szerez egy SSH szolgáltatáson keresztül, akkor akár parancsokat is futtathat a rendszeren, ezért nagyon fontos ennek a szolgáltatásnak a védelme és megfelelő korlátozása.

A honeypot egy olyan csapdarendszer, amely valós szolgáltatásnak látszik, de valójában megfigyelési és naplózási célt szolgál. Egy SSH honeypot például úgy viselkedik, mintha egy valódi SSH szerver lenne, de közben rögzíti, hogy milyen IP-címről érkezett a kapcsolat, milyen felhasználóneveket és jelszavakat próbáltak ki, illetve sikeres belépés után milyen parancsokat adtak ki. Ez oktatási és biztonsági szempontból is hasznos, mert valósághű adatokat ad a támadási mintákról.

Fontos szempont volt, hogy a honeypot ne veszélyeztesse a valódi rendszert. Emiatt a projektet Proxmox alatt, külön virtuális gépeken alakítottam ki, és a kommunikációt tűzfalszabályokkal, valamint elkülönített belső hálózattal korlátoztam. Így a tesztelés során keletkező kapcsolatok csak a laborban lévő gépek között történtek, nem pedig külső hálózatról.

2. Shell emuláció és proxy mód közötti különbség

A projekt elején felmerült, hogy a Cowrie-t Shell emulációs módban használjam. Ebben az esetben a támadó egy Cowrie által szimulált parancskörnyezetet lát, tehát nem egy valódi Linux gépen dolgozik, hanem egy emulált rendszeren. Ez a megoldás egyszerűbb, könnyebben beállítható és biztonságosabb abból a szempontból, hogy nincs mögötte valódi backend gép. Hátránya viszont, hogy kevésbé valóságos, mivel a parancsok és a fájlrendszer csak szimulálva vannak.

A proxy mód ezzel szemben úgy működik, hogy a támadó a Cowrie-hoz kapcsolódik, de a Cowrie a háttérben egy valódi backend Linux géphez továbbítja a kapcsolatot. A támadó ebből azt érzékeli, mintha egy valódi SSH szerverre jelentkezett volna be, miközben a Cowrie köztes réteggént naplózza az eseményeket. Ebben a felépítésben a Cowrie látja és rögzíti a bejelentkezéseket, a parancsokat és a session adatokat, de a parancskörnyezetet már a backend gép adja.

Végül a proxy mód mellett döntöttem, mert a projekt célja egy valóságosabb honeypot környezet kialakítása volt. A backend gépen létre tudtam hozni olyan könyvtárakat, mint például a projects és a backups, amelyek egy valódi szerver hatását keltették. Ezáltal a támadó szempontjából a rendszer hitelesebbnek tűnik, miközben a Cowrie továbbra is képes naplózni az eseményeket.

3. Anyag és módszer

3.1. Anyag

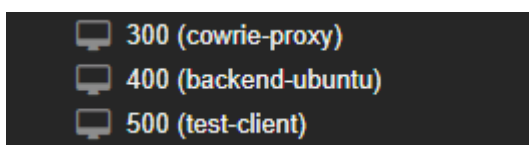
A projektben az alábbi komponensek játszottak fő szerepet:

1. **Proxmox**: Egy virtualizációs platform, amelyen a VM-ek futottak.
2. **UFW**: Az UFW egy egyszerű Linux tűzfalkezelő eszköz. A projektben ezzel korlátoztam, hogy melyik gép melyik port-ot érheti el.
3. **Authbind**: Lehetővé teszi, hogy nem root felhasználó is használjon alacsony port-okat, például a 22-es port-ot. A projektben ezzel tudott a Cowrie a standard SSH port-on futni.
4. **Cowrie**: Egy ezen a néven futó projekt, amely lehetővé teszi az SSH port-on történő támadások naplózását.
5. **Három darab virtuális gép**: cowrie-proxy (10.50.0.10/24), test-client (10.50.0.20/24), backend-ubuntu (10.50.0.30/24)

3.2. Módszer

3.2.1. Hálózati infrastruktúra

Első próbálkozásban, mivel Shell emulációt akartam csinálni először csak kettő VM-et telepítettem egyet, mint támadó és egyet, mint a cowrie proxy gépe. Azonban miután úgy döntöttem, hogy nem egy emulációt készítek, hanem a proxy módot használom, hozzá kellett adnom egy backend gépet is, így összesen 3 virtuális gépet hasznosítottam a megvalósítás során.



1. ábra A három gép Proxmox felületén

Fontos részét képviselte az a projektben, hogy mivel egy minél valóság közelebb megvalósításra törekedtem, nem szerettem volna egy esetleges tényleges támadás áldozatává válni. Első lépésben előkészítettem, hogy a számítógépek minél hamarabb le legyenek vágva az internet felől és egy teljesen steril környezetben konfiguráljam őket. Létrehoztam egy extra Linux bridge-t, aminek nem volt hozzáférése az internethez és hozzáadtam a gépek hálózati kártyái közé, mint elérés. Ezután a netplan állományát szerkesztve hozzáadtam az

extra Linux bridge konfigurációját, de az internet elérésével rendelkezőt nem tiltottam le, amíg a megfelelő csomagokat le nem töltöttem a gépekre.

```
GNU nano 7.2
network:
  version: 2
  ethernet:
    ens18:
      dhcp4: true
    ens19:
      dhcp4: false
  addresses:
    - 10.50.0.10/24
```

Network Device (net0)	virtio=BC:24:11:7F:80:C3,bridge=vbr0,firewall=1,link_down=1
Network Device (net1)	virtio=BC:24:11:92:2D:FD,bridge=vbr1,firewall=1

2. ábra Hardver és gép konfiguráció

Miután a csomagokat letöltöttem, mint hardver komponens a proxmox segítségével kikapcsoltam az internet felé nyitott hálózati kártyát, majd a netplanba is kikapcsoltam az adott kártya DHCP címozását. Így az egyetlen kommunikációs sáv csakis a másik hálózati kártyán és csakis a három gép között történhet. Utolsó lépésben beállítottam az UFW segítségével a tűzfalszabályokat a gépeken, hogy a cowrie proxy SSH-n elérje a backendet, de például a test-client ne érje el.

```
sudo ufw default deny incoming
sudo ufw default deny outgoing
sudo ufw allow from 10.50.0.20 to any port 22 proto tcp
sudo ufw allow out to 10.50.0.30 port 22 proto tcp
```

3.2.2. Cowrie SSH Honeypot telepítése és konfigurálása

A Cowrie SSH honeypot konfigurálását a cowrie-proxy nevű gépen végeztem el. A cél az volt, hogy a Cowrie a test-client felől valódi SSH szolgáltatásnak tűnjön, ezért a megszokott 22-es port-on figyeljen, miközben proxy módban egy külön backend gépre továbbítja a kapcsolatot.

A Cowrie-t nem a fő admin felhasználóval futtattam, hanem egy külön cowrie nevű Linux felhasználóval. Ez biztonsági szempontból fontos, mert így a honeypot folyamat nem root jogosultsággal futott. A Cowrie projekt a GitHub oldaláról letölthető. A futtatásához szükség van egy Python környezetre is.

A konfiguráció nagy része a projektben /home/cowrie/cowrie/etc/cowrie.cfg elérésű fájlban zajlott. Ez a fájl felel a beállítások nagy részéért itt lehet az emuláció, proxy és egy ilyen honeypot szerver farm-szerű beállítás közül választani. Az én esetemben a proxy módot választottam, mivel a szerver farmok, amelyek a legvalóságosabbak, az én esetemben nem kivitelezhetőek. Át kellett még állítanom az alapbeállításról a port figyelést a 2222 port-ról a 22-es port-ra, hogy valóságos legyen a feladatom. Majd megadtam a backend-ubuntu adatait, hogy milyen IP címen található és melyik felhasználóba jelentkezzen be a cowrie kapcsolatlétesítésekor. Éppen ezért a backend-ubuntu nevű gépen létrehoztam egy

```
backend = simple

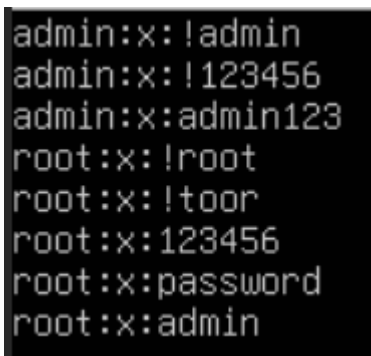
# =====
# Simple Backend Configuration
# =====
backend_ssh_host = 10.50.0.30
backend_ssh_port = 22

backend_telnet_host = localhost
backend_telnet_port = 2023
```

3. ábra Cowrie konfigurációs fájl

felhasználót, aki nem rendelkezik admin jogosultságokkal. Szoktak ilyenkor különböző hamis fontos fájlokat alkalmazásokat és egyébeket is létrehozni ezen a felhasználón, hogy minél hihetőbb legyen ez a hamis gép.

A `/home/cowrie/cowrie/etc/userdb` egy másik igencsak fontos fájl, amely a különböző tiltott, illetve elfogadott felhasználónév és jelszó kombinációkat tartalmazza, mellyel egy támadó sikeresen vagy sikertelen behatolhat. Ennek a formátuma felhasználónév:x:jelszó ez a formátum számos az informatika világában jól ismert jelölést is felismer, mint például a `'!'` mint tagadás, azaz egy nem elfogadott jelszó és felhasználónév páros vagy a `'*'` jel amely egy bármely kifejezést testesít meg. Én az ábrán látható beállításokat használtam.



```
admin:x:!admin
admin:x:!123456
admin:x:admin123
root:x:!root
root:x:!toor
root:x:123456
root:x:password
root:x:admin
```

4. ábra Userdb tartalma

Ezek után még egy nagyon fontos beállításra volt szükség. Meg kellett valósítani, hogy a 22-es port-on való elérés működjön. Ehhez számos lehetőség volt, így sok időt töltöttem a megfelelő eszköz kiválasztásában. Fontolóra vettem a port tűzfallal történő átirányítását, iptables vagy nftables segítségével, viszont ez a lehetőség el lett vetve, mivel attól tartottam, hogy csak belebonyolódnék. Az sem szólt mellette, hogy ekkor már az UFW szabályok meglettek adva.

Egy másik megoldást a setcap jelentette, amelynek az a lényege, hogy egy program vagy bináris külön jogosultságot kap arra, hogy alacsony port-ot nyisson. Ilyenkor például a Python vagy a Cowrie indításához használt bináris kaphatná meg a `CAP_NET_BIND_SERVICE` képességet. Így szintén elkerülhettem volna a Cowrie root-ként való futtatását. Hátránya, hogy Python alapú alkalmazásnál ez kényesebb és kevésbé tiszta, mert nem mindig egyértelmű, pontosan melyik binárisra kell adni a jogosultságot. Ráadásul túl széles jogosultságot is adhattam volna egy általános Python binárisnak, amit más programok is használhatnak. Emiatt végül elvetettem ezt az ötletet és az authbind mellett döntöttem.

Az authbind azáltal oldotta meg ezt a problémát, hogy nem az egész program kapott root jogosultságot, hanem csak egy nagyon konkrét engedélyt adtam: a cowrie felhasználó használhatta a 22-es port-ot. Tehát a Cowrie továbbra is sima, korlátozott Linux user-ként futott, de képes volt megnyitni az SSH alapértelmezett port-ját.

Megvalósításban is igen egyszerű volt, hiszen telepítés után csupán a /etc/authbind/byport/ útvonalon belül kellett egy 22 nevű fájlt létrehozni, majd átadni a fájl tulajdonosi jogát a cowrie felhasználónak és read, execute jogokat adni.

Ezek után a lépések után csupán a tesztelés volt hátra. Az ábrán látható parancs sorozattal lehetett a cowrie-t elindítani.

```
sudo su - cowrie
cd /home/cowrie/cowrie
source cowrie-env/bin/activate
export PYTHONPATH=/home/cowrie/cowrie/src
AUTHBIND_ENABLED=yes cowrie start
```

5. ábra Cowrie indítási lépései

3.2.3. Naplóadatok kinyerése és feldolgozása

A rendszer működésének ellenőrzése után a projekt következő lépése a Cowrie által rögzített adatok kinyerése és feldolgozása volt. Ennek célja az volt, hogy a honeypot ne csak technikailag működjön, hanem értékelhető adatokat is szolgáltatson a tesztelt SSH-kapcsolatokról, a bejelentkezési próbálkozásokról és a kiadott parancsokról. Az elemzéshez nem külső előfizetési rendszert használtam, hanem a Cowrie saját naplófájljait dolgoztam fel egy egyszerű, átlátható Python alapú megoldással.

A tesztelést a külön erre létrehozott test-client virtuális gépről végeztem. Ez a gép a támadó szerepét töltötte be a laborhálózatban, és csak a Cowrie proxy gép irányába próbált SSH-kapcsolatot létesíteni. A cél nem valódi támadás végrehajtása volt, hanem olyan események létrehozása, amelyek alapján ellenőrizhető, hogy a honeypot megfelelően rögzíti-e a sikertelen és sikeresnek látszó bejelentkezéseket.

A teszt során több különböző felhasználónév és jelszó párost próbáltam ki. A sikertelen belépések azt mutatták meg, hogy a Cowrie képes naplózni a rossz hitelesítési kísérleteket, míg a beállított tesztfelhasználóval történő sikeres belépés után azt is ellenőrizni tudtam, hogy a rendszer a parancsokat és a munkamenetet is rögzíti. A bejelentkezés után egyszerű, tipikusan támadói felderítéshez hasonló parancsokat futtattam, például a könyvtárak listázását, az aktuális felhasználó lekérdezését, a rendszerinformációk megtekintését és a hamis projekt- vagy mentési könyvtárak tartalmának vizsgálatát.

A backend gépen szándékosan olyan könyvtárak és fájlnevek is szerepeltek, amelyek valós környezet benyomását keltik. Ilyenek voltak például a projects és backups könyvtárak. Ezek nem valódi érzékeny adatokat tartalmaztak, hanem a honeypot hihetőségét növelték, hogy egy belépő támadó ne egy teljesen üres, mesterséges környezetet lásson.

A Cowrie a naplókat a cowrie-proxy gépen, a /home/cowrie/cowrie/var/log/cowrie könyvtárban tárolta. A legfontosabb állomány a cowrie.json volt, mert ez soronként egy-egy JSON formátumú eseményt tartalmaz. Emellett a cowrie.log emberileg könnyebben olvasható formában mutatta a működés közbeni eseményeket és hibákat.

A JSON napló azért volt hasznosabb az elemzéshez, mert szerkezetben tartalmazta az időbélyeget, az eseménytípust, a forrás IP-címet, a munkamenet azonosítóját, a felhasználónevet, a jelszót és a kiadott parancsokat. Emiatt később automatizáltan lehetett belőle Excel-táblázatot készíteni, szemben a sima szöveges naplóval, amelyet nehezebb egységesen feldolgozni.

```
{ } cowriejson > ...
1 { "eventid": "cowrie.session.connect", "src_ip": "10.50.0.20", "src_port": 38194, "dst_ip": "10.50.0.10", "dst_port": 22, "sessio
2 { "eventid": "cowrie.client.version", "version": "SSH-2.0-OpenSSH_9.6p1 Ubuntu-3ubuntu13.15", "message": "Remote SSH version
3 { "eventid": "cowrie.proxy.backend_connected", "backend_ip": "10.50.0.30", "backend_port": 22, "local_ip": "10.50.0.10", "loca
4 { "eventid": "cowrie.client.kex", "hassh": "aae6b9604f6f3356543709a376d7f657", "hasshAlgorithms": "sntrup761x25519-sha512@op
5 { "eventid": "cowrie.login.failed", "username": "root", "password": "123456", "message": "login attempt [root/123456] failed",
6 { "eventid": "cowrie.login.failed", "username": "root", "password": "password", "message": "login attempt [root/password] fail
7 { "eventid": "cowrie.proxy.backend_disconnected", "backend_ip": "10.50.0.30", "backend_port": 22, "local_ip": "10.50.0.10", "lo
```

6. ábra JSON fájl kinézete

A naplófájlokat először a cowrie-proxy gépen gyűjtöttem össze. A szükséges állományokat egy exportcsomagba másoltam, majd tömörítettem, hogy egyszerűbben lehessen őket áthelyezni a tesztkliensre, illetve később a saját Windows gépre. A gyakorlatban ez azt jelentette, hogy a cowrie.json és a cowrie.log fájlokból egy cowrie-export.tar.gz csomag készült.

A fájlátvitel során figyelni kellett arra, hogy a laborhálózat tűzfalszabályai eredetileg szigorúan korlátozták a gépek közötti kommunikációt. A Cowrie proxy és a backend között csak a szükséges SSH-forgalom volt engedélyezve, a test-client pedig alapvetően a támadó szerepét töltötte be. Emiatt az exportálásnál ellenőrizni kellett, hogy az adott irányú kapcsolat valóban engedélyezett-e, illetve, hogy a fogadó gépen fut-e SSH szolgáltatás. Ahol a közvetlen átmásolás nem volt megfelelő, ott a Proxmox host-on keresztüli USB-s átvitel jelentette a gyakorlati megoldást.

Az átvitel után a naplócsomag a Windows gépre került, ahol a fájlokat kicsomagoltam, és a feldolgozáshoz a cowrie.json állományt használtam fel. Ez a módszer egyszerű maradt,

de mégis megőrizte az eredeti naplókat, így szükség esetén később is visszaellenőrizhető, hogy az Excelben szereplő adatok honnan származnak.

Az adatok feldolgozásához Python szkriptet készítettem Visual Studio Code környezetben. A szkript a cowrie.json fájlt soronként olvasta be, mivel a Cowrie JSON naplója nem egyetlen nagy JSON tömb, hanem JSON Lines formátumú állomány. Ez azt jelenti, hogy minden sor külön eseményként értelmezhető.

A program a beolvasott eseményeket különböző szempontok szerint rendezte. Külön kezelte a bejelentkezési próbálkozásokat, a sikeres munkameneteket, a parancskiadásokat, valamint a nyers eseményeket. Az Excel-fájl létrehozásához az openpyxl könyvtárat használtam, mert ez lehetővé tette, hogy több munkalapból álló, később kézzel is szerkeszthető .xlsx állomány készüljön.

timestamp	src_ip	session	command	message
2026-05-02T11:13:37.057337Z	10.50.0.20	1b2a42be5c14	whoami	CMD: whoami
2026-05-02T11:13:40.286817Z	10.50.0.20	1b2a42be5c14	hostname	CMD: hostname
2026-05-02T11:13:45.646898Z	10.50.0.20	1b2a42be5c14	pwd	CMD: pwd
2026-05-02T11:13:46.887506Z	10.50.0.20	1b2a42be5c14	ls	CMD: ls
2026-05-02T11:13:52.430152Z	10.50.0.20	1b2a42be5c14	ls -la	CMD: ls -la
2026-05-02T11:14:05.935932Z	10.50.0.20	1b2a42be5c14	cat /etc/os-release	CMD: cat /etc/os-release
2026-05-02T11:14:13.108503Z	10.50.0.20	1b2a42be5c14	uname -a	CMD: uname -a
2026-05-02T11:14:24.253191Z	10.50.0.20	1b2a42be5c14	cd projects/	CMD: cd projects/
2026-05-02T11:14:29.712821Z	10.50.0.20	1b2a42be5c14	ls	CMD: ls
2026-05-02T11:14:36.128963Z	10.50.0.20	1b2a42be5c14	cat todo.txt	CMD: cat todo.txt
2026-05-02T11:14:46.311776Z	10.50.0.20	1b2a42be5c14	cd ../backups	CMD: cd ../backups
2026-05-02T11:14:51.001001Z	10.50.0.20	1b2a42be5c14	ls	CMD: ls
2026-05-02T11:14:59.163945Z	10.50.0.20	1b2a42be5c14	c[1Pat targets.txt	CMD: c[1Pat targets.txt
2026-05-02T11:15:02.352299Z	10.50.0.20	1b2a42be5c14	exit	CMD: exit

7. ábra Parancsok

A feldolgozás közben felmerült egy gyakorlati hiba is: néhány naplózott parancsban olyan vezérlőkarakter szerepelt, amelyet az Excel nem tudott közvetlenül cellába írni. Emiatt a szkriptbe bekerült egy tisztító lépés, amely a problémás karaktereket eltávolította vagy biztonságos szöveggé alakította. Ezután a program már hibamentesen tudta létrehozni az Excel állományt.

Az elkészült Excel táblázat célja nem pusztán a naplófájl átmásolása volt, hanem az adatok áttekinthetőbbé tétele. A munkafüzetben külön munkalap készült az összes eseménynek, a bejelentkezési próbálkozásoknak, a kiadott parancsoknak és az összesítő adatoknak. Így az elemzés során gyorsan meg lehetett nézni, hogy milyen

timestamp	eventid	src_ip	username	password	session	message
2026-05-02T11:12:16.725585Z	cowrie.login.failed	10.50.0.20	root	123456	f993def61202	login attempt [root/123456] failed
2026-05-02T11:12:27.141478Z	cowrie.login.failed	10.50.0.20	root	password	f993def61202	login attempt [root/password] failed
2026-05-02T11:12:47.131459Z	cowrie.login.failed	10.50.0.20	ubuntu	password	b4d01dcdc5c8	login attempt [ubuntu/password] failed
2026-05-02T11:12:59.666876Z	cowrie.login.failed	10.50.0.20	backup	admin	f2d58e256b8c	login attempt [backup/admin] failed

8. ábra Belépések

felhasználónevekkel próbálkoztak, milyen jelszavakat használtak, hány munkamenet jött létre, és milyen parancsokat adtak ki a sikeres belépés után.

Az összesítő részben a legfontosabb statisztikák szerepeltek. Ilyen volt például az események teljes száma, a bejelentkezési kísérletek száma, a leggyakrabban használt felhasználónevek, a leggyakoribb jelszavak, valamint a kiadott parancsok gyakorisága. Ez a forma azért volt előnyös, mert a dokumentációhoz és a bemutatóhoz is könnyen felhasználható eredményeket adott.

Mutató	Érték
Összes esemény	52
Egyedi sessionök száma	5
Egyedi forrás IP-k száma	1
Parancs események száma	14
Sikertelen login próbák száma	6
Sikeres login próbák száma	1

9. ábra Összegzés

A projekt során felmerült a Splunk használata is, mivel naplóelemzésre és biztonsági események áttekintésére jól használható eszköz. Végül ezt nem választottam, mert a free trial jelleg és az előfizetési modell nem illett jól a projekt egyszerű, oktatási célú környezetéhez.

Alternatívaként szóba került a Grafana és Loki használata is. Ez technikailag jó irány lehetett volna, mert látványosabb grafikonokat és folyamatos naplógyűjtést tenne lehetővé. Ugyanakkor a beállítása több plusz komponenst igényelt volna, például Docker, Loki, Grafana és esetleg Alloy konfigurációt. Mivel a projekt fő célja a Cowrie proxy honeypot működésének bemutatása volt, nem szerettem volna, hogy a naplóelemző infrastruktúra beállítása elvigye a hangsúlyt a honeypot működéséről.

Ezért a végső megoldás a naplók exportálása és Python segítségével Excelbe rendezése lett. Ez kevésbé professzionális, mint egy teljes SIEM vagy központi loggyűjtő rendszer, de a projekt keretei között átlátható, reprodukálható és könnyen bemutatható eredményt adott.

A tesztelési és elemzési rész alapján ellenőrizhetővé vált, hogy a Cowrie proxy honeypot nemcsak fogadja a tesztelt SSH-kapcsolatokat, hanem részletesen naplózza is a fontos eseményeket. A test-client segítségével létrehozott bejelentkezési próbálkozások és parancsok megjelentek a cowrie.json fájlban, majd ezekből strukturált Excel állomány készült.

timestamp	eventid	src_ip	src_port	dst_ip	dst_port	username
2026-05-02T11:12:03.990590Z	cowrie.session.connect	10.50.0.20	38194	10.50.0.10	22	
2026-05-02T11:12:03.992562Z	cowrie.client.version	10.50.0.20				
2026-05-02T11:12:03.994511Z	cowrie.proxy.backend_connected	10.50.0.20				
2026-05-02T11:12:03.995725Z	cowrie.client.kex	10.50.0.20				
2026-05-02T11:12:16.725585Z	cowrie.login.failed	10.50.0.20				root
2026-05-02T11:12:27.141478Z	cowrie.login.failed	10.50.0.20				root
2026-05-02T11:12:31.767727Z	cowrie.proxy.backend_disconnected	10.50.0.20				

10. ábra Esemény naplózás

Az elkészült feldolgozás lehetővé tette, hogy a projekt eredményeit ne csak képernyőképekkel, hanem konkrét, táblázatos adatokkal is be lehessen mutatni. A megoldás külön előnye, hogy nem igényelt további szerveroldali logelemző rendszert, mégis alkalmas volt a honeypot által gyűjtött információk értelmezésére és dokumentálására.

4. Hibák és tanulságok

A projekt során több olyan probléma is előjött, amelyből fontos tanulságokat lehetett levonni. Az egyik legnagyobb nehézséget az okozta, hogy a rendszer több különálló részből állt, és ezek beállításai egymásra épültek. A Proxmox hálózati beállításai, a netplan YAML fájlok, az UFW tűzfalszabályok, a Cowrie konfigurációja, az authbind használata és a backend gép SSH elérése mind külön-külön is hibaforrást jelentettek. Ha ezek közül egy elem rosszul volt beállítva, akkor a teljes kapcsolat hibásan működött.

Tanulásként levonható, hogy egy ilyen rendszert érdemes előre részletesebben megtervezni. Hasznos lett volna már a projekt elején pontosan rögzíteni, hogy melyik gép milyen IP-címet kap, melyik portot használja, melyik irányba engedélyezett a kommunikáció, és melyik gépen milyen szolgáltatásnak kell futnia. A munka közben több alkalommal utólag derült ki, hogy egy csomag hiányzik, egy szolgáltatás nem fut, vagy egy tűzfalszabály nem megfelelő irányba engedélyezi a kapcsolatot. Ezeket később sikerült javítani, de a folyamat ettől szétszórtabbá vált.

Külön problémát jelentett a Cowrie 22-es port-ra való beállítása is. Mivel a 22-es port alacsony számú, privilegizált port, ezért egy normál felhasználóként futó alkalmazás nem tudja közvetlenül használni. Erre végül az authbind megoldást választottam, amely lehetővé tette, hogy a Cowrie ne rootként fusson, de mégis képes legyen a 22-es port-on figyelni. Ez biztonsági szempontból jobb megoldás volt, mint root jogosultsággal futtatni az egész honeypotot.

A Cowrie konfigurációjánál szintén előjött hiba abból, hogy bizonyos beállítások több helyen is szerepelhetnek. A proxy módnál figyelni kellett arra, hogy a backendhez tartozó felhasználónév és jelszó a megfelelő konfigurációs részben legyen megadva, és ne legyenek duplikált opciók. A hibakeresés során látható volt, hogy egy rosszul elhelyezett vagy ismétlődő beállítás miatt a Cowrie el sem tud indulni.

A naplók feldolgozásánál is jelentkeztek kisebb problémák. A cowrie.json fájl soronként külön JSON eseményeket tartalmazott, ezért nem klasszikus egyetlen nagy JSON objektumként kellett kezelni. Emellett az Excelbe írásnál bizonyos karakterek hibát okoztak, mert az openpyxl nem enged minden vezérlőkaraktert cellába írni. Ebből az következik, hogy a naplófeldolgozásnál mindig szükség van adattisztításra, különösen akkor, ha parancsokat vagy felhasználói bemeneteket akarunk táblázatos formába exportálni.

Összességében a legfontosabb tanulság az volt, hogy egy honeypot rendszer nemcsak egy program telepítéséből áll. A működéshez hálózati tervezés, jogosultságkezelés,

tűzfalbeállítás, szolgáltatásellenőrzés, naplózás és adatelemzés is szükséges. A projekt során ezek mind megjelentek, ezért a hibák ellenére a rendszer jó alapot adott annak megértésére, hogyan épül fel egy többgépes, proxy alapú SSH honeypot környezet.

5. Továbbfejlesztési lehetőségek

A projekt továbbfejlesztésének egyik lehetséges iránya egy komolyabb megfigyelő és naplóelemző rendszer bevezetése lenne. Eredetileg felmerült a Splunk használata is, de ennek licencelése és előfizetési jellege miatt végül nem ezt választottam. Alternatívaként szóba jöhetne a Grafana, Loki és Alloy alapú megoldás, amely ingyenes és rugalmasabb irány lehetne. Ezzel a naplók nemcsak Excelben lennének feldolgozva, hanem webes felületen, grafikonokkal és kereshető naplókval is megjelenhetnének.

A Grafana és Loki használata azonban összetettebb telepítést igényelne. Ehhez valószínűleg Docker konténerekre, Alloy vagy más loggyűjtő komponensre, valamint külön konfigurációkra lenne szükség. Ez a projekt jelenlegi szintjén túl sok plusz hibakeresést jelenthetett volna, de továbbfejlesztési irányként jó lehetőség. Egy ilyen rendszerrel könnyebben lehetne kimutatásokat készíteni például a leggyakoribb felhasználónevekről, jelszavakról, IP-címekről, session időkről és parancsokról.

További fejlesztési lehetőség lenne más honeypotok bevonása is. A Cowrie elsősorban SSH és Telnet honeypotként használható, de egy összetettebb honeypot környezetben más szolgáltatásokat is lehetne szimulálni. Például webes honeypot, FTP honeypot vagy adatbázis honeypot is bekerülhetne a rendszerbe. Így nemcsak SSH próbálkozásokat lehetne vizsgálni, hanem többféle támadási irányt is.

Egy fejlettebb megoldásban honeypot clustert is lehetne kialakítani. Ebben több különböző honeypot gép működne egyszerre, különböző szolgáltatásokkal és eltérő konfigurációkkal. A naplók egy központi elemző gépre kerülnének, ahol egységes formában lehetne őket vizsgálni. Ez közelebb állna egy valós biztonsági megfigyelő rendszerhez, ahol több forrásból érkeznek események egy központi naplózó vagy SIEM rendszerbe.

A backend gép kezelését is lehetne automatizálni. Proxy mód esetén a támadó egy valódi Linux környezetbe kerül, ezért fontos, hogy a backend rendszer ne maradjon hosszú ideig módosított állapotban. Továbbfejlesztésként megoldható lenne automatikus backend tisztítás, például időzített visszaállítással, snapshot használatával vagy script segítségével. Így minden session után vagy naponta vissza lehetne állítani a backend gépet egy ismert, tiszta állapotra.

A jelenlegi Excel alapú feldolgozás is továbbfejleszthető lenne. A Python script kiegészíthető lenne több statisztikával, külön diagramokkal, automatikus összesítő munkalapokkal és szűrhető táblázatokkal. Így a dokumentációhoz és a prezentációhoz

közvetlenül lehetne használni a generált eredményeket. Ez egyszerűbb irány, mint egy teljes Grafana rendszer telepítése, de mégis látványosabb és rendezettebb elemzést adna.

Végül érdemes lenne a projektet úgy továbbfejleszteni, hogy a rendszer indítás után automatikusan működőképes állapotba kerüljön. Ez jelenthetné a Cowrie automatikus indulását, az authbind beállítások ellenőrzését, a tűzfalszabályok betöltését és a naplóexport egyszerűsítését. Így a rendszer kevésbé függene kézi parancsoktól, és stabilabban lehetne újraindítás után is használni.

6. Összegzés

A projekt során egy Cowrie alapú SSH honeypot rendszert alakítottam ki Proxmox virtualizációs környezetben. A cél egy olyan tesztkörnyezet létrehozása volt, amelyben biztonságosan lehet szimulált SSH bejelentkezési próbálkozásokat végezni, majd ezek eredményét naplózni és elemezni. A rendszer nem éles internetes támadások gyűjtésére készült, hanem oktatási és demonstrációs célból, elkülönített belső hálózaton működött.

A megvalósítás során három fő virtuális gépet használtam. A cowrie-proxy gépen futott maga a Cowrie honeypot, a backend-ubuntu gép szolgáltatta a valósabbnak tűnő háttérrendszert, míg a test-client gépről történtek a szimulált belépési kísérletek. A hálózati kommunikáció úgy lett kialakítva, hogy a test-client a Cowrie gép 22-es port-ját érje el, a Cowrie pedig proxy módban továbbítsa a kapcsolatot a backend gép felé. Így a támadó szemszögéből a rendszer egy valódi SSH szervernek tűnik, miközben a Cowrie köztes réteggként rögzíti az eseményeket.

A projekt egyik fontos döntése az volt, hogy nem a Cowrie egyszerű Shell emulációs módját használtam, hanem a proxy módot. Ez több beállítást és több hibalehetőséget jelentett, viszont valósághűbb környezetet adott. A backend gépen létrehozott könyvtárak és felhasználói környezetek miatt a belépő fél számára a rendszer kevésbé tűnt üres vagy mesterséges gépnek. A Cowrie közben naplózta a kapcsolódásokat, a bejelentkezési próbálkozásokat, a kiadott parancsokat és a session eseményeket.

A naplófájlokat végül a Cowrie gépről kinyertem, majd a test-client-en és később a saját gépen dolgoztam fel. A cowrie.json fájl tartalmazta a strukturált eseményeket, amelyeket Python segítségével Excel formátumba alakítottam. Ez lehetővé tette, hogy a naplózott adatok áttekinthetőbb formában legyenek elemezhetők, például IP-címek, felhasználónevek, jelszavak, sessionök és kiadott parancsok alapján. Így a projekt nemcsak a honeypot működését mutatta be, hanem azt is, hogyan lehet az összegyűjtött adatokat feldolgozható formába rendezni.

Ábrajegyzék

1. ábra A három gép Proxmox felületén.....	6
2. ábra Hardver és gép konfiguráció.....	7
3. ábra Cowrie konfigurációs fájl	8
4. ábra Userdb tartalma.....	9
5. ábra Cowrie indítási lépései.....	10
6. ábra JSON fájl kinézete	11
7. ábra Parancsok.....	12
8. ábra Belépések.....	12
9. ábra Összegzés.....	13
10. ábra Esemény naplózás.....	14